



Harbinger: Classical Change-Point Methods



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Roadmap

This tutorial series walks through methods for detecting structural changes in time series, progressing from the simplest case to the most specialized. The journey begins with the intuition of a single breakpoint and advances toward more nuanced analytical perspectives, including volatility modeling and local window contrast.

01

Single Breakpoint

AMOC — the core concept of regime transition

02

Multiple Breakpoints

BinSeg and PELT — recursive and optimal segmentation

03

Structural Tests

Chow and GFT — bridging change-point detection with classical statistical testing

04

ChangeFinder

LR, ARIMA, ETS — measuring predictive surprise as it unfolds over time

05

Volatility and Windows

GARCH and SCP — capturing variance dynamics and local contrast

Single Breakpoint – AMOC

AMOC (**At Most One Change**) is the foundational change-point detection method. It finds the single most statistically significant structural break by minimizing a cost function over all possible split points.

Mathematical Intuition

$$\hat{\tau} = \arg \min_{\tau} [C(x_{1:\tau}) + C(x_{\tau+1:n})]$$

The optimal split point $\hat{\tau}$ marks the location of the single most significant structural break.

Objective

Detect the single most significant change point by globally minimizing a cost function over all candidate split locations

Assumptions

The series contains **at most one breakpoint**; not suited for data with multiple structural changes

Strengths

Simple, interpretable, and **computationally cheap** — $O(n)$ scan; ideal as a pedagogical baseline

Limitations

Fails with multiple breaks. **Sensitive** to cost function choice, which can shift breakpoint location

Further Reading: Hinkley, D.V. (1970). Inference about the change-point in a sequence of random variables. *Biometrika*, 57(1), 1–17. — The seminal paper establishing the statistical foundation for single change-point inference.

Preparing the Data

```
# Instalar Harbinger (se necessário)
#install.packages("harbinger")

# Carregar pacotes necessários
library(daltoolbox)
library(harbinger)

# Carregar datasets de change-point
data(examples_changepoints)

# Selecionar dataset "complex"
# (contém múltiplos regimes)
dataset <- examples_changepoints$complex
head(dataset)

## serie event
## 1 0.3129618 FALSE
## 2 0.5944808 FALSE
## 3 0.8162731 FALSE
```

Step-by-Step Walkthrough

- Install `harbinger` once before first use
- Load `daltoolbox` and `harbinger` at the start of every session
- Use `data(examples_changepoints)` to access built-in datasets
- Select the `complex` dataset, which spans multiple distinct regimes
- Inspect with `head()` — check the `serie` and `event` columns

 **Key Concept:** Always examine the raw signal before fitting any model — skipping this step can hide regime structure and make results harder to interpret.

Visualize the Series and Configure the Model

```
# Visualizar a série temporal
har_plot(harbinger(), dataset$serie)

# Configurar o detector AMOC
# (uma única mudança dominante)
model <- hcp_amoc()

# Ajustar o detector
# (sem treinamento necessário;
# mantém parâmetros no objeto)
model <- fit(model, dataset$serie)
```

Step-by-step

- `har_plot()` displays the raw series before any fitting
- Visually inspect the plot to spot likely regime transitions
- `hcp_amoc()` instantiates the detector with no required parameters
- `fit()` anchors parameters in the model object, preparing it for detection

📌 **Key concept:** AMOC searches all possible breakpoint locations to find the single most significant change point.

Detect, Evaluate, and Plot

```
# Executar detecção na série completa
detection <- detect(model, dataset$serie)
```

```
# Exibir índices detectados
print(detection |>
  dplyr::filter(event == TRUE))
```

```
## idx event type
## 1 389 TRUE changepoint
```

```
# Avaliar contra eventos rotulados
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
```

```
## event
## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  4 495
```

```
# Plotar detecções sobre a série
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step

- `detect()` returns a data frame with an `event` column marking detected indices
- Use `dplyr::filter(event == TRUE)` to isolate detected change points
- AMOC flagged index **389** as the single most significant change point
- `evaluate()` compares detections to true labels via a confusion matrix
- `har_plot()` with four arguments overlays detections and ground truth on the series

❏ **Key concept:** Treat scores as evidence of how well the method's assumptions hold in practice, not as abstract numbers.

Multiple Breakpoints – Binary Segmentation (BinSeg)

BinSeg is a **greedy recursive algorithm** that extends AMOC to detect multiple structural changes, recursively splitting segments until a maximum number of breakpoints **Q** or a penalty threshold is reached.

Mathematical Intuition

At each step, find the optimal split point by minimizing the total cost across both segments:

$$\tau^* = \arg \min_{\tau} [\mathcal{C}(x_{1:\tau}) + \mathcal{C}(x_{\tau+1:n})]$$

Split at τ^* , then repeat independently on each sub-segment until the stopping criterion is met.

✓ Strengths

- **Fast:** $O(n \log n)$ — scales well to large series
- **Widely used** across many domains
- **Easy to implement;** builds directly on AMOC

⚠ Limitations

- **Greedy:** early splits are never revisited
- **Not globally optimal:** no guarantee of best segmentation
- **Sensitive to Q:** max breakpoints strongly influences results

📘 **Further Reading:** Vostrikova, L.Yu. (1981). Detecting "disorder" in multidimensional random processes. *Soviet Mathematics Doklady*, 24, 55–59.

Recursive Binary Segmentation

```
# Configurar BinSeg
# Q = número máximo de change points
model <- hcp_binseg(Q = 10)
model <- fit(model, dataset$serie)

# Executar detecção
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 101 TRUE changepoint
## 2 200 TRUE changepoint
## 3 312 TRUE changepoint
## 4 327 TRUE changepoint
```

```
## 5 349 TRUE changepoint
## 6 368 TRUE changepoint
## 7 389 TRUE changepoint

# Avaliar e plotar
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   6
##  FALSE  3 490

har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-Step Walkthrough

- `hcp_binseg(Q = 10)` sets a maximum of 10 change points to search for
- The algorithm recursively splits the series around the largest detected structural change
- A total of **7 change points** were detected at indices 101, 200, 312, 327, 349, 368, and 389
- The confusion matrix reveals 1 true positive and 6 false positives, highlighting the risk of over-detection
- Use `har_plot()` to visually verify how detected breaks align with the actual event structure

📌 **Key concept:** BinSeg is a heuristic — it can over-detect. The `Q` parameter controls the upper bound on the number of breakpoints the algorithm will seek.

Multiple Breakpoints – Optimal Partitioning (PELT)

PELT is an exact change-point detection algorithm that finds the globally optimal segmentation of a time series under a penalized cost framework. Unlike BinSeg, it guarantees the globally optimal set of breakpoints while achieving near-linear runtime through a pruning step.

Mathematical Intuition

PELT minimizes the penalized total cost over all possible segmentations:

$$\text{minimize over all } \tau : \sum \text{ of } \mathcal{C}(x_{\tau_{i-1}+1:\tau_i}) + \beta \cdot m$$

where $\mathcal{C}$ is the within-segment cost, β is the penalty per breakpoint, and m is the number of breakpoints. The pruning rule eliminates candidate points τ whenever:

$$\mathcal{C}(x_{\tau:t}) + \beta \geq \mathcal{C}(x_{\tau:s}) + \mathcal{C}(x_{s:t}) \quad \text{for all future } t$$

This ensures linear expected runtime under mild distributional assumptions.

1

✓ Strengths

- Globally optimal segmentation (not greedy)
- Near-linear $O(n)$ expected runtime
- Penalty β controls complexity — higher $\beta \rightarrow$ fewer breakpoints

2

⚠ Limitations

- Optimality depends on penalty choice — ill-suited β causes over- or under-detection
- Assumes a cost function appropriate for the data distribution

Further Reading: Killick, R., Fearnhead, P., & Eckley, I.A. (2012). Optimal detection of changepoints with a linear computational cost. Journal of the American Statistical Association, 107(500), 1590–1598.

Optimal Partitioning with Pruning

```
# Usar dataset "simple"
dataset <- examples_changepoints$simple

# Configurar e ajustar PELT
model <- hcp_pelt()
model <- fit(model, dataset$serie)

# Executar detecção
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 9 TRUE changepoint
## 2 19 TRUE changepoint
## 3 29 TRUE changepoint
## 4 39 TRUE changepoint
```

```
## 5 60 TRUE changepoint
## 6 71 TRUE changepoint
## 7 81 TRUE changepoint
## 8 91 TRUE changepoint

# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE 0 8
##   FALSE 1 92
```

Step-by-step

- Use the `simple` dataset for a cleaner, well-controlled test case
- `hcp_pelt()` requires no mandatory parameters — the default penalty is applied automatically
- PELT detected **8 change points**, but none aligned with the true labels
- The confusion matrix shows 0 true positives and 8 false positives on this dataset
- This highlights how strongly the choice of penalty influences detection results

📌 **Key concept:** PELT guarantees optimality under the chosen penalty, but an ill-suited penalty can lead to over-detection even with linear-time complexity.

Summary: Method Families and When to Use Them

Classical – AMOC

`hcp_amoc()` — Use when **at most one** structural break is expected. Minimizes a cost function over all split points. Simple, fast, and interpretable.

Classical – BinSeg

`hcp_binseg(Q)` — Use when **multiple breakpoints** are expected and speed matters. Greedy recursive splits; set Q as the upper bound on breakpoints.

Classical – PELT

`hcp_pelt()` — Use when **globally optimal** segmentation is required. Penalized cost minimization with near-linear runtime; tune β to control sensitivity.

Statistical Tests – Chow & GFT

`hcp_chow()` / `hcp_gft()` — Use when a **hypothesis-testing framework** is preferred. Chow tests a known candidate point; GFT scans all locations via F-statistics.

ChangeFinder – LR, ARIMA, ETS

`hcp_cf_lr()` / `hcp_cf_arima()` / `hcp_cf_ets()` — Use when changes manifest as **shifts in predictive surprise**. Fits a model and flags points where forecast error spikes.

Volatility & Windows – GARCH & SCP

`hcp_garch()` / `hcp_scp()` — Use when changes occur in **variance dynamics** (GARCH) or are best detected by **local window contrast** (SCP).

Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.

References

Hinkley, D.V. (1970). Inference about the change-point in a sequence of random variables. *Biometrika*, 57(1), 1–17.

Vostrikova, L.Yu. (1981). Detecting "disorder" in multidimensional random processes. *Soviet Mathematics Doklady*, 24, 55–59.

Killick, R., Fearnhead, P., & Eckley, I.A. (2012). Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500), 1590–1598.

Zeileis, A., Leisch, F., Hornik, K., & Kleiber, C. (2002, 2003). strucchange: An R package for testing structural change. *Journal of Statistical Software*, 7(2).

Takeuchi, J., & Yamanishi, K. (2006). A unifying framework for detecting outliers and change points from time series. *IEEE Transactions on Knowledge and Data Engineering*.

Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer.